



SEMT30008 Week 20 Lab: Large Language Model Pretraining

Introduction

Welcome to the LLM Pretraining Lab!

In this 2-hour session, we will demystify the "black box" of modern large language models. Based on our lecture, we will implement the core components of the 4 pillars of LLM pretraining:

- Data Pipeline (Cleaning, Deduplication, Tokenization)
- Objectives & Architecture (Causal Masking, RMSNorm, SwiGLU)
- Training Dynamics & Infra (Learning Rates, Memory, BF16)
- Utilizing Base Models (Prompting)

Let's dive in! Run the setup cell below to import necessary libraries.

```
In [ ]: # Run this first!  
import re  
import math  
import torch  
import torch.nn as nn  
import numpy as np  
import matplotlib.pyplot as plt  
!pip install transformers -q  
from transformers import AutoTokenizer
```

Module 1: The Data Pipeline (The "Dirty Work")

Reference: Slides 6-19

Garbage In, Garbage Out! A model is only as good as its data. Common Crawl drops 80%-90% of raw bytes during processing. Let's implement some of these crucial steps.

Exercise 1: PII Removal (Decontamination)

To prevent privacy leaks, we must mask Personally Identifiable Information (PII) like emails and phone numbers.

Task: Write a Regex function to mask emails and phone numbers with `[EMAIL]` and `[PHONE]`.

```
In [ ]: def remove_pii(text):
        # TODO 1: Write a regex pattern to replace emails with '[EMAIL]'
        text = re.sub(r'__', '[EMAIL]', text)
        # TODO 2: Write a regex pattern to replace phone numbers (e.g., 123-456-78
        text = re.sub(r'__', '[PHONE]', text)
        return text

raw_text = "Contact Mengyue at mengyue@llm-course.edu or call 555-019-2024 for
print(remove_pii(raw_text))
```

Exercise 2: Fuzzy Deduplication (Jaccard Similarity)

If an LLM sees a text 1000 times, it memorizes it. Let's implement Jaccard Similarity, the math behind MinHash LSH used for fuzzy deduplication.

Task: Calculate the Jaccard similarity between two token sets.

```
In [ ]: def jaccard_similarity(doc1, doc2):
        set1, set2 = set(doc1.split()), set(doc2.split())
        # TODO 3: Calculate intersection and union
        intersection = __
        union = __
        return len(intersection) / len(union)

d1 = "The quick brown fox jumps over the dog"
d2 = "The quick brown fox jumps over the lazy dog"
print(f"Jaccard Similarity: {jaccard_similarity(d1, d2):.2f}")
```

Exercise 3: Tokenization (Subword-level)

Neural networks process numbers, not text. Let's see how modern Byte-Pair Encoding (BPE) breaks down a tricky word.

Task: Load a standard HuggingFace tokenizer and tokenize the word "Unbelievable!".

```
In [ ]: # TODO 4: Load the "gpt2" tokenizer using AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("___")

text = "Unbelievable!"
tokens = tokenizer.tokenize(text)
token_ids = tokenizer.encode(text)

print(f"Tokens: {tokens}")
print(f"Token IDs: {token_ids}")
```

Exercise 4: Modern Considerations (Handling Numbers)

Older models fragment numbers destroying arithmetic logic. Modern models force individual digit tokenization.

Task: Write a preprocessing function that spaces out numbers so the tokenizer treats them individually.

```
In [ ]: def force_digit_tokenization(text):
        # TODO 5: Use regex to insert a space between consecutive digits
        # Hint: you might need a loop or re.sub with groups
        return ____

math_prompt = "Calculate 12345 + 67"
processed_prompt = force_digit_tokenization(math_prompt)
print(f"Original Tokens: {tokenizer.tokenize(math_prompt)}")
print(f"Digit Tokens: {tokenizer.tokenize(processed_prompt)}")
```

Module 2: Objectives & Architecture

Reference: Slides 20-38

The original 2017 Transformer is obsolete. We will implement the crucial upgrades: Causal Masking, RMSNorm, and SwiGLU.

Exercise 5: Decoder-Only Causal Masking

Without masking, self-attention allows token i to "cheat" by looking at token $i+1$.

Task: Create a 5×5 causal mask matrix where future tokens are $-\infty$ and past/current are 0.

```
In [ ]: def create_causal_mask(seq_len):
        # TODO 6: Create a matrix of ones, then use torch.tril to get the lower tr
        mask = torch.ones(seq_len, seq_len)
        mask = torch.tril(____)

        # TODO 7: Convert 0s to float('-inf') and 1s to 0.0
        causal_mask = mask.masked_fill(mask == 0, ____ )
        causal_mask = causal_mask.masked_fill(mask == 1, ____ )
        return causal_mask

print(create_causal_mask(5))
```

Exercise 6: RMSNorm (Root Mean Square Normalization)

LayerNorm centers the mean (slow!). RMSNorm drops mean centering entirely, making it 10%-50% faster.

Task: Implement the RMSNorm equation: $\frac{x}{\sqrt{\text{Mean}(x^2) + \epsilon}}$

```
In [ ]: class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x):
        # TODO 8: Calculate the mean of the square of x along the last dimension
        norm_x = x.pow(2).mean(dim=-1, keepdim=True)
        # TODO 9: Normalize x and multiply by the learnable weight
        x_normed = x * torch.rsqrt(norm_x + self.eps)
        return self.weight * x_normed

dummy_input = torch.randn(2, 4) # batch=2, dim=4
rmsnorm = RMSNorm(4)
print(rmsnorm(dummy_input))
```

Exercise 7: SwiGLU Activation Function

SwiGLU yields better perplexity across all model scales. It uses a gating mechanism: $\text{Swish}(x W_1) \otimes (x W_2)$.

Task: Implement the forward pass of a SwiGLU block.

```
In [ ]: class SwiGLU(nn.Module):
    def __init__(self, dim_in, dim_out):
        super().__init__()
        self.w1 = nn.Linear(dim_in, dim_out)
        self.w2 = nn.Linear(dim_in, dim_out)
        self.w3 = nn.Linear(dim_out, dim_in)

    def forward(self, x):
        # TODO 10: Apply the SwiGLU formula: Swish(xW1) * xW2
        # Note: PyTorch's silu() is equivalent to Swish
        gate = torch.nn.functional.silu(self.w1(x))
        up = self.w2(x)
        return self.w3(gate * up)

swiglu_layer = SwiGLU(128, 256)
out = swiglu_layer(torch.randn(1, 128))
print("Output shape:", out.shape) # Should be [1, 128]
```

Exercise 8: Grouped-Query Attention (GQA) Memory Calculation

Standard Multi-Head Attention (MHA) consumes massive GPU memory for KV Cache.

Task: Calculate the number of KV heads in MHA, MQA, and GQA (Llama-style).

```
In [ ]: num_query_heads = 32

# TODO 11: Set the correct number of KV heads for each architecture
mha_kv_heads = ____ # Multi-Head Attention
mqa_kv_heads = ____ # Multi-Query Attention
gqa_kv_heads = num_query_heads // 8 # Grouped-Query Attention (assuming 8 Q heads)

print(f"MHA KV Heads: {mha_kv_heads}")
print(f"MQA KV Heads: {mqa_kv_heads}")
print(f"GQA KV Heads: {gqa_kv_heads}")
```

Module 3: Scaling Laws

Reference: Slides 39-43

Training a 70B model costs millions. We cannot guess hyperparameters. We rely on the "physics" of AI.

Exercise 9: The Chinchilla Paradigm

DeepMind found that to be Compute-Optimal, Model Parameters (N) and Data Tokens (D) must scale equally. Golden Ratio: ~ 20 tokens per parameter.

Task: Calculate optimal training tokens for a given model size.

```
In [ ]: def calculate_chinchilla_tokens(parameters_in_billions):
# TODO 12: Implement the Chinchilla rule ( $D \approx 20N$ )
tokens_in_billions = ____
return tokens_in_billions

gpt3_params = 175 # Billion
print(f"GPT-3 should have been trained on {calculate_chinchilla_tokens(gpt3_params)} tokens")
# Reality check: GPT-3 was only trained on 300B tokens. It was severely undertrained
```

Exercise 10: The Llama Era (Overtraining)

Meta ignored Chinchilla to optimize for inference speed, training a small model (8B) on a massive dataset (15 Trillion tokens).

Task: Calculate the token-per-parameter ratio for Llama-3-8B.

```
In [ ]: llama3_params = 8_000_000_000 # 8 Billion
        llama3_tokens = 15_000_000_000_000 # 15 Trillion

        # TODO 13: Calculate the actual token-to-parameter ratio for Llama 3
        ratio = ____

        print(f"Llama-3 Ratio: {ratio} tokens per parameter.")
        print(f"This is {ratio / 20:.1f}x times the Chinchilla limit!")
```

Module 4: Training Dynamics & Infra

Reference: Slides 44-53

Exercise 11: Learning Rate (LR) Scheduling

If LR is too high -> Explodes (NaN). If LR is too low -> Stops learning.

Task: Implement a Cosine Decay learning rate function with a minimum LR of 10%.

```
In [ ]: def cosine_decay_lr(step, total_steps, max_lr):
        min_lr = max_lr * 0.10 # 10% of max_lr
        # TODO 14: Implement the cosine decay mathematical formula
        # Formula: min_lr + 0.5 * (max_lr - min_lr) * (1 + cos(pi * step / total_s
        decay_ratio = step / total_steps
        coeff = 0.5 * (1.0 + math.cos(math.pi * decay_ratio))
        return min_lr + coeff * (max_lr - min_lr)

        steps = np.arange(0, 1000)
        lrs = [cosine_decay_lr(s, 1000, 3e-4) for s in steps]
        plt.plot(steps, lrs)
        plt.title("Cosine Decay LR Schedule")
        plt.xlabel("Training Steps")
        plt.ylabel("Learning Rate")
        plt.show()
```

Exercise 12 & 13: The Memory Bottleneck (BF16 vs FP16)

FP16 has a max value of ~65,504. LLM gradients easily exceed this causing NaN (Loss Spikes). BF16 solves this by trading fraction bits for exponent bits.

Task: Let's force an FP16 overflow in PyTorch to see why BF16 is the standard today.

```
In [ ]: # TODO 15: Create a tensor with a value of 50,000 in standard FP32
        tensor_fp32 = torch.tensor([50000.0], dtype=torch.float32)

        # Multiply by 2
```

```

result_fp32 = tensor_fp32 * 2
print(f"FP32 Result: {result_fp32.item()}")

# TODO 16: Cast tensor_fp32 to FP16 and multiply by 2
tensor_fp16 = tensor_fp32.to(____)
result_fp16 = tensor_fp16 * 2
print(f"FP16 Result (Overflow!): {result_fp16.item()}")

# TODO 17: Cast tensor_fp32 to BF16 (bfloat16) and multiply by 2
tensor_bf16 = tensor_fp32.to(____)
result_bf16 = tensor_bf16 * 2
print(f"BF16 Result (Safe!): {result_bf16.item()}")

```

Exercise 14: Distributed Training Memory Math

A single H100 GPU has 80GB of VRAM. Let's prove mathematically why a 70B parameter model requires Multi-GPU 3D Parallelism.

Task: Calculate the total VRAM required for just the parameters and AdamW optimizer states in BF16 (2 bytes per parameter).

```

In [ ]: params = 70_000_000_000 # 70B

# TODO 18: Calculate bytes needed for model weights in BF16 (2 bytes per param
weight_bytes = params * ____

# TODO 19: AdamW stores Momentum and Variance for EVERY parameter, also in BF1
# How many bytes is this? (2 states * 2 bytes * params)
adam_bytes = params * ____

total_gb = (weight_bytes + adam_bytes) / (1024**3)
h100_count = math.ceil(total_gb / 80)

print(f"Total memory needed: {total_gb:.2f} GB")
print(f"Minimum H100 80GB GPUs required: {h100_count}")

```

Module 5: Utilizing the Base Model

Reference: Slides 54-58

Exercise 15: Prompt-Based Zero-Shot Learning

Base models don't naturally answer questions; they just predict the next word. We must format our tasks as "Proxy Tasks".

Task: Write a Python function that wraps a raw movie review into a prompt that tricks the Base Model into generating the sentiment class ("Positive" or

"Negative").

```
In [ ]: def format_sentiment_prompt(review_text):  
        # TODO 20: Use an f-string to append a trigger phrase (e.g. " The sentiment  
        # that encourages the LLM to output 'Positive' or 'Negative'  
        prompt = f"Review: '{review_text}'\nQuestion: Is the sentiment of this review  
        return prompt  
  
        review = "The cinematography was breathtaking, but the plot was a complete mess"  
        print(format_sentiment_prompt(review))
```